

CSCI 682 - Topics in Artificial Intelligence:

Natural Language Processing

Assignment 3

Solutions to the questions on this assignment should be submitted via PDF and .py files to Canvas before **May 3rd at 11:59 pm**. Make sure to justify your answers.

I highly encourage you to collaborate with one another. However, you must write up your own solutions **independently**. Feel free to communicate via [Discord](#) and to post questions on the appropriate forum in Canvas. Do not post solutions. Also, please include a list of the people you work with at the top of your submission.

Have fun!

Problems

1. In this question, we will implement a hidden Markov model to perform named entity recognition on the [CoNLL-2003](#) dataset and compare its performance against independent token classifiers. The dataset is available in Canvas.
 - (a) (3 pts) Write Python code to load and preprocess the dataset. The data should already be split into `training`, `validation`, and `test` sets. Replace words that do not appear in the [Gensim glove-wiki-gigaword-100](#) embedding with an `<UNK>` token and include tags for the start and end of sentences. Briefly describe any additional preprocessing that you perform. Include code snippets of the relevant functions in your PDF submission.
 - (b) (10 pts) Implement a first-order hidden Markov model for BIO tagging. Using the training data, estimate transition probabilities between labels and emission probabilities from labels to words with add-1 smoothing. Then implement the Viterbi algorithm to decode the most likely label sequence for a sentence in the test data. Include code snippets of the relevant functions in your PDF submission.
 - (c) (10 pts) Train four token classifiers to predict BIO labels without conditioning on neighboring predicted labels, using sparse features from [DictVectorizer](#) and dense embeddings from [glove-wiki-gigaword-100](#). In particular, implement
 - i. logistic regression using [scikit-learn](#) with sparse features using [DictVectorizer](#).
 - ii. logistic regression using [scikit-learn](#) with dense word embeddings.
 - iii. a neural network with one hidden layer accepting the embedding of a single word as input.

- iv. a neural network with one hidden layer accepting the embeddings of the previous word, current word, and next word as input.
- For each model, add at least two additional features that do not depend on classifications of other words (e.g. capitalization) and explain your reasoning. Discuss how you determined values for relevant hyperparameters.
- (d) (7 pts) Evaluate all four models on the test set and compare performance. Show a confusion matrix for each approach and compute relevant metrics. Discuss your results and consider examples of each approach outperforming the others.
2. Consider the implementation of a transition-based dependency parser developed with the assistance of OpenAI's Codex available in `assignment_3_q_2_skeleton.py`. In this problem, we will use the `ud_english_ewt` data, available on Canvas, to train a classifier that predicts the next transition given current parser state.
 - (a) (10 pts) Write a function to extract features associated with the state of the parser. At a minimum, include the words and part-of-speech tags of the top two items on the stack and the first two items in the buffer. You may include additional features of your choice. Include code snippets of the relevant functions in your PDF submission.
 - (b) (7 pts) Use `simulateGoldTransitions` and `collectTransitionExamples` to construct supervised training examples from gold-standard dependency trees. Train a classifier to predict the next parser transition from the current parser state using the implementation of logistic regression from `scikit-learn` and `DictVectorizer`. Include code snippets of the relevant functions in your PDF submission.
 - (c) (8 pts) Use your classifier together with the provided greedy decoder to parse the test sentences. Use the provided `parseSentenceSplit` and `evaluateParses` functions to generate predicted dependency trees and compute UAS and exact match. Discuss the parser's performance.
 3. Extra Credit - Explore dependency trees in [Universal Dependencies](#).
 - (a) (5 pts) As an alternative to the transition-based dependency parser from question 2, consider a graph-based dependency parser. For each possible dependency arc $u \rightarrow v$ in a sentence, define a score $score(u, v)$ using a classifier or linear model trained on head-dependent pairs from the training data. Then select a maximum weight spanning tree. See section 19.3 of the textbook for more details on this approach.
 - (b) (5 pts) Analyze the dependency trees for several languages. Consider comparing number of leaves (sentence length), tree depth, and a simple structural similarity measure such as edge-overlap similarity.
 - (c) (5 pts) Train a Word2Vec-like embedding using the [Gensim](#) implementation or your own implementation on dependency tree data. In particular, instead of considering positive examples based on a window around a given word, focus on connections in the tree. So (u, v) is a positive example if there is an edge (regardless of direction) between u and v in a sentence. Compare the resulting embedding to a more typical Word2Vec embedding. How do they differ? How are they the same?